使用 Agent2Agent 的多代理系統

來源：https://codelabs.developers.google.com/next26/adk-agent2agent?hl=zh-tw

步驟數：12

使用 Agent2Agent (A2A) 通訊協定建構多代理系統。

目錄

步驟 1 · 約 2 分鐘

1. 總覽

在本程式碼實驗室中，您將建構多代理系統，讓多個 ADK 代理使用 Agent2Agent (A2A) 通訊協定進行通訊及協作。

課程內容

- 如何建立多個獨立的 ADK 代理
- 如何為每個代理提供代理資訊卡，並將其包裝為 A2A 伺服器
- 如何建構主機代理程式來調度遠端代理程式
- 如何建立遠端代理程式連線
- 如何在本機測試多代理系統

軟硬體需求

- 已啟用計費功能的 Google Cloud 雲端專案
- 網路瀏覽器，例如 Chrome
- Python 3.12 以上版本

本程式碼研究室適合對 Python 和 Google Cloud 有基本認識的中階開發人員。

完成這個程式碼研究室大約需要 15 分鐘。

本程式碼研究室建立的資源費用應低於 $5 美元。

步驟 2 · 約 2 分鐘

2. 設定環境

> **Google Cloud 抵免額：**

建立 Google Cloud 專案

1. 在 Google Cloud 控制台的專案選取器頁面中，**選取或建立 Google Cloud 專案**。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何檢查專案是否已啟用計費功能。

啟動 Cloud Shell 編輯器

如要從 Google Cloud 控制台啟動 Cloud Shell 工作階段，請在 Google Cloud 控制台中點選「啟用 Cloud Shell」。

系統便會在 Google Cloud 控制台的底部窗格啟動工作階段。

如要啟動編輯器，請點選 Cloud Shell 視窗工具列中的「開啟編輯器」。

設定環境

首先，請在終端機中執行下列指令，為 A2A 系統建立專案資料夾結構。在本示範中，我們會使用 $HOME 目錄的絕對路徑：

```
mkdir -p $HOME/app/src/agents $HOME/app/src/host
touch $HOME/app/.env $HOME/app/pyproject.toml
```

現在我們已瞭解一般架構，接下來請填入環境設定。將下列程式碼片段複製到新的 `.env` 檔案中：

在新的 `.env` 檔案中填入 GCP 專案 ID 和 GCP 區域。如要從即將建立的代理程式接收報表電子郵件，也可以在 `MAIL_TO` 中輸入所選電子郵件。此外，您也可以新增 GitHub 個人存取權杖 `GITHUB_TOKEN`，方便 GitHub 擷取代理程式運作。

使用下列 bash 指令開啟新的 .env 檔案：

```
cloudshell edit .env
```

然後將下列檔案複製到 `app.env` 的 .env 檔案中。重要注意事項：請務必輸入您自己的值。

```
# --- Google Cloud ---
GOOGLE_GENAI_USE_VERTEXAI=TRUE
GOOGLE_CLOUD_PROJECT=<your-gcp-project-id>
GOOGLE_CLOUD_LOCATION=<your-gcp-project-region>

# --- GitHub ---
# Personal Access Token with "repo" scope
# Create one at: https://github.com/settings/tokens
# Generate new token --> Fine-grained, repo-secured --> (populate) Token name --> (scroll
down) Generate token
GITHUB_TOKEN=<your-github-pat>
MCP_SERVER_HOST=https://api.githubcopilot.com/mcp/
TARGET_REPOS=["google/adk-python", "langchain-ai/langchain"]
DEFAULT_ISSUE_COUNT=5
DEFAULT_PR_COUNT=5

# --- Email (Optional) ---
# Only needed if you want the emailer agent to send reports
RESEND_API_KEY=<your-resend-API-key>
RESEND_DOMAIN=<your-domain>
MAIL_TO=<insert-email-to-receive-reports>

# --- Local Development Overrides  ---
RETRIEVAL_AGENT_URL=http://localhost:8001
EVALUATOR_AGENT_URL=http://localhost:8002
EMAILER_AGENT_URL=http://localhost:8003
ORCHESTRATOR_PORT=http://localhost:8000
```

現在您已填入環境變數，接下來必須設定 uv 環境。將下列程式碼片段複製到 `pyproject.toml` 檔案中：

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "app"
version = "0.1.0"
description = "Multi-agent system designed to assess the newest issues and pull requests from
numerous agentic development platforms"
authors = [
    { name = "Thomas Wagner" },
    { name = "Kris Overholt" }
]
license = "Apache-2.0"
requires-python = ">=3.11,<3.14"
dependencies = [
    "resend>=2.21.0",
    "uvicorn>=0.40.0",
    "a2a-sdk>=0.3.26,<0.4.0",
    "google-adk[a2a]>=1.27.0,<1.30.0",
    "google-generativeai>=0.8.4",
    "httpx>=0.28.1",
    "pydantic>=2.12.5, <3.0.0",
    "python-dotenv>=1.2.0",
    "nest-asyncio>=1.6.0",
]

[project.optional-dependencies]
test = [
    "pytest>=8.3.2",
    "pytest-asyncio>=0.23.8",
    "pytest-mock>=3.14.0",
    "respx>=0.21.0",
]

[tool.ruff]
target-version = "py313"
line-length = 80

[tool.ruff.lint]
select = ["E", "F", "I", "C", "PL", "B", "UP", "RUF"]
ignore = ["E501", "C901"]

[tool.ruff.lint.per-file-ignores]
"__init__.py" = ["F401"]

[tool.ruff.lint.isort]
known-first-party = ["src"]

[tool.hatch.build.targets.wheel]
packages=["src/"]
```

建立虛擬環境

現在，請在終端機中，從您剛建立的 `app` 目錄執行下列 bash 指令碼。這會設定 Python 虛擬環境，並從 `pyproject.toml` 檔案安裝所有必要依附元件。

```
# Ensure you are in the 'app' directory before running this

# Exit immediately if a command exits with a non-zero status.
set -e

# 1. Install uv, the Python package manager used for this project
echo "Installing uv..."
if ! command -v uv &> /dev/null
then
    pip install uv
else
    echo "uv is already installed."
fi

# 2. Create and activate virtual environment
echo "Setting up virtual environment..."
# --clear ensures you start with a fresh environment
uv venv --clear

# 3. Install dependencies
echo "Installing Python dependencies..."
uv sync

echo "Installation and setup complete."
```

現在我們已準備好建立多代理系統！

步驟 3 · 約 1 分鐘

3. 建立 Retriever 代理程式

Eva 是軟體開發人員，希望在 GitHub 存放區更新新問題和 PR 時，隨時掌握最新資訊。因此，她建立 ADK 代理程式，用於擷取她要求的 GitHub 資料。

為了方便進行這個範例，請從專案根層級執行下列指令，為檢索器代理程式建立必要的目錄和檔案：

```
mkdir -p $HOME/app/src/agents/retriever
touch $HOME/app/src/agents/retriever/agent.py
touch $HOME/app/src/agents/retriever/__init__.py
echo "from . import agent" >> $HOME/app/src/agents/retriever/__init__.py
```

將下列 ADK 程式碼片段複製到 `$HOME/app/src/agents/retriever/agent.py`：

```
import logging
import os
import json
from dotenv import load_dotenv
from google.adk.agents.llm_agent import Agent
from google.adk.tools.mcp_tool.mcp_session_manager import (
    StreamableHTTPConnectionParams,
)
from google.adk.tools.mcp_tool.mcp_toolset import McpToolset

logger = logging.getLogger(__name__)
load_dotenv()

GITHUB_MCP_URL = os.getenv(
    "GITHUB_MCP_URL", "https://api.githubcopilot.com/mcp/"
)
GITHUB_TOKEN = os.getenv("GITHUB_TOKEN")
if GITHUB_TOKEN is None:
    raise ValueError("GITHUB_TOKEN env is not set")
TARGET_REPOS = os.getenv("TARGET_REPOS")
DEFAULT_ISSUE_COUNT = os.getenv("DEFAULT_ISSUE_COUNT")
DEFAULT_PR_COUNT = os.getenv("DEFAULT_PR_COUNT")

# --- Prompt ---
GITHUB_RETRIEVAL_INSTRUCTIONS = f"""
    You are a specialized GitHub data retrieval agent.
    Your only purpose is to fetch data from GitHub using the available MCP tools and format
it for another agent.

    **DEFAULT CONFIGURATION:**
    If the orchestrator does not specify which repositories or how many items to fetch, you
MUST use the following defaults:
    - **Repositories:** {TARGET_REPOS}
    - **PR Count:** {DEFAULT_PR_COUNT} per repository
    - **Issue Count:** {DEFAULT_ISSUE_COUNT} per repository

    **Your Task:**
    1. Analyze the task. If no specific repo is mentioned, iterate through the Default
Repositories list above.
    2. Use the `GitHub` MCP tool to fetch the requested data (Pull Requests and/or Issues).
    3. **CRITICAL:** After the tool has finished running, you MUST take the raw output and
compile it into a single response.

    The orchestrator is waiting for this raw data to pass to the Evaluator. Do not summarize
it.
"""


# --- Agent Initialization ---
root_agent = Agent(
    model="gemini-2.5-flash",
    name="retriever",
```

```
    instruction=GITHUB_RETRIEVAL_INSTRUCTIONS,
    description="""
        Connects to the GitHub MCP server to retrieve real-time development
        insights, actively reading repository issues, pull requests, and commit
        histories.
    """,
    tools=[
        McpToolset(
            connection_params=StreamableHTTPConnectionParams(
                url=GITHUB_MCP_URL,
                headers={
                    "Authorization": f"Bearer {GITHUB_TOKEN}",
                    "X-MCP-Toolsets": "repos,issues,pull_requests",
                    "X-MCP-Readonly": "true",
                },
            )
        )
    ],
)
```

這個代理程式可做為獨立工具使用。如要獨立測試，請執行下列指令，在 **/agents/ 目錄中執行 uv run adk web：

```
cd $HOME/app/src/agents

uv run adk run retriever
```

在「uv run adk web」終端機中開啟 localhost 連結，然後選取要試用的代理程式。

步驟 4 · 約 1 分鐘

4. 建立評估人員代理程式

Richard 經常需要向非技術人員客戶和同事解釋許多技術術語。Richard 厭倦了需要定義相同術語，並以精簡的方式說明相同專案，因此製作了蒸餾代理程式，將技術術語歸納為容易理解的文字。

為了方便進行這個試用版，請執行下列指令，為評估人員代理建立檔案：

```
mkdir -p $HOME/app/src/agents/evaluator
touch $HOME/app/src/agents/evaluator/agent.py
touch $HOME/app/src/agents/evaluator/__init__.py
echo "from . import agent" >> $HOME/app/src/agents/evaluator/__init__.py
```

將下列代理程式碼片段複製到 `$HOME/app/src/agents/evaluator/agent.py` 中

```
import json
from google.adk.agents.llm_agent import Agent

# --- Prompt ---
EVAL_AND_SUMMARIZATION_INSTRUCTIONS = """
    You are a specialized analysis and summarization agent. Your only purpose is to take raw,
structured text about GitHub repositories and transform it into a concise, human-readable
Markdown report.

    **Your Task:**
    1. You will receive a block of text containing raw data about pull requests and issues
from an orchestrator.
    2. Analyze the provided data. For pull requests, evaluate their significance. For issues,
identify key themes and problems.
    3. **CRITICAL:** You MUST generate a comprehensive summary in Markdown format based on
your analysis. Your final output should be ONLY this Markdown report. Do not add any
conversational text or explanations (e.g., "Here is the summary..."). The orchestrator needs
to pass your clean report to another agent or directly to the user.

    **Output Format Rules:**
    - The report MUST be in Markdown.
    - Structure the report by repository.
    - For each repository, provide a concise overview of significant pull requests and
important issues.
    - Conclude with overall insights.

    The orchestrator is waiting for this report. Ensure your final response consists of
nothing but the complete Markdown summary.
"""

# --- Agent ---
root_agent = Agent(
    model="gemini-2.5-flash",
    name="evaluator",
    instruction=EVAL_AND_SUMMARIZATION_INSTRUCTIONS,
    description="""
        Distills and summarizes complex GitHub data, breaking down pull
        requests, code changes, and issue discussions into easily understandable
        insights.
    """,
)
```

這個代理程式可做為獨立工具使用。如要獨立測試，請在「新」終端機中執行下列指令，在 **/agents/ 目錄中執行 uv run adk web：

```
cd $HOME/app/src/agents

uv run adk run evaluator
```

開啟終端機中的 localhost 連結，然後選取評估人員代理程式進行試用。

步驟 5 · 約 1 分鐘

5. 建立電子郵件代理程式

伊凡經常需要撰寫電子郵件，但內容只是摘要並重新排版容易取得的文字，這讓他感到厭煩。因此他製作了電子郵件代理程式，可重新格式化指定文字，並傳送至提供的電子郵件帳戶。

為了方便進行這個試用版，請在終端機中執行下列指令，為電子郵件代理建立檔案：

```
mkdir -p $HOME/app/src/agents/emailer
touch $HOME/app/src/agents/emailer/agent.py
touch $HOME/app/src/agents/emailer/__init__.py
echo "from . import agent" >> $HOME/app/src/agents/emailer/__init__.py
```

將下列代理程式碼片段複製到 `app/src/agents/emailer/agent.py`

```
import logging
import os
import resend

from dotenv import load_dotenv
from google.adk.agents.llm_agent import Agent

load_dotenv()
logger = logging.getLogger(__name__)

RESEND_API_KEY = os.getenv("RESEND_API_KEY")
RESEND_DOMAIN = os.getenv("RESEND_DOMAIN")
MAIL_TO = os.getenv("MAIL_TO")

if RESEND_API_KEY:
    resend.api_key = RESEND_API_KEY

# --- Tools ---
def send_report_email(recipient: str, subject: str, body: str) -> str:
    """
    Sends an email of the given subject and body to the specified recipient
    via Resend. If recipient is None, it defaults to the MAIL_TO environment variable.

    Args:
        recipient (str | None): The email address of the recipient. If None, defaults to
MAIL_TO.
        subject (str): The subject of the email.
        body (str): The body of the email.

    Returns:
        str: A success or failure message.
    """
    if not recipient:
        recipient = MAIL_TO

    if not all([RESEND_API_KEY, RESEND_DOMAIN, recipient]):
        error_msg = "Error: Email tool configuration missing (API Key, Domain, or Recipient)"
        logger.error(error_msg)
        return error_msg

    try:
        html_body = f"<div style='font-family: sans-serif; white-space: pre-wrap;'>{body}
</div>"

        params: resend.Emails.SendParams = {
            "from": f"Research Agent <agent@{RESEND_DOMAIN}>",
            "to": [recipient], #type: ignore
            "subject": subject,
            "text": body,
            "html": html_body,
        }
```

```
        email = resend.Emails.send(params)

        logger.info(f"Email sent successfully. ID: {email['id']}")
        return f"Email sent! ID: {email['id']}"

    except Exception as e:
        error_msg = f"Failed to send email: {e}"
        logger.error(error_msg)
        return error_msg

# --- Prompt ---
EMAIL_INSTRUCTIONS = """
    You are an emailer agent responsible for formatting and sending a research report via
email.

    INPUT: You will receive a Markdown-formatted string (the report) and the email recipient.

    YOUR TASK:
    1. Take the Markdown content and format it appropriately for an email body.
       The goal is to render the Markdown effectively so it is readable and well-presented in
an email client.
    2. Based on the summary, generate a concise and informative subject line for the email.
       The subject line should reflect the main themes or key insights from the report.
    3. Use the `send_report_email` tool with the extracted recipient, the generated subject
line, and the formatted email body.
       If no email recipient is provided, output a message indicating that no recipient was
specified and do NOT call the tool.
"""

# --- Agent ---
root_agent = Agent(
    model="gemini-2.5-flash",
    name="emailer",
    instruction=EMAIL_INSTRUCTIONS,
    description="""
        Acts as the final delivery mechanism in the pipeline, dispatching
        generated summaries and reports to designated email addresses.
    """,
    tools=[
        send_report_email
    ],
)
```

這個代理程式可做為獨立工具使用。如要獨立測試，請在「新」終端機中執行下列指令，在 **/agents/ 目錄中執行 uv run adk web：

```
cd $HOME/app/src/agents

uv run adk run emailer
```

開啟終端機中的 localhost 連結，然後選取要試用的代理程式。

步驟 6 · 約 1 分鐘

6. 建立代理商資訊卡

我們剛才介紹了三位獨立開發人員的故事，他們都有自己獨特的代理商。所有這些代理程式都會發布，並登入公司代理程式庫。另一位開發人員 Diane 想將這些個別想法整合到單一多代理系統中。

她的目標是擁有隨選研究代理程式，隨時掌握不同代理程式開發架構的問題和 PR。她也希望這項工具能以電子郵件傳送生態系統中所有新發展的摘要。由於所有代理程式都是在不同環境中個別開發，因此 A2A 是整合這些現有代理程式的理想解決方案。

如要組裝一系列遠端代理程式，第一步是為每個必要遠端代理程式提供代理程式卡。這些就像是代理的商務名片。這些 JSON 檔案可讓主機代理程式依名稱、說明、功能和輸入/輸出結構定義識別代理程式。

首先，請從專案根目錄執行下列指令，建立 `cards` 目錄和代理程式資訊卡所需的所有 JSON 檔案：

```
mkdir -p $HOME/app/cards/
touch $HOME/app/cards/github_retrieval_agent_card.json
touch $HOME/app/cards/content_evaluator_agent_card.json
touch $HOME/app/cards/emailer_agent_card.json
```

這些檔案只能透過下列 bash 指令存取：

```
cloudshell edit $HOME/app/cards/github_retrieval_agent_card.json
```

將下列 JSON 內容複製到 `app/cards/github_retrieval_agent_card.json` 資料夾。可存取這個檔案的對象：

```
{
  "name": "GitHub_Retrieval_Agent",
  "description": "Connects to the GitHub MCP server to retrieve real-time development
insights, actively reading repository issues, pull requests, and commit histories.",
  "url": "http://localhost:8001",
  "capabilities": {
    "streaming": true,
    "pushNotifications": true,
    "stateTransitionHistory": false
  },
  "defaultInputModes": [
    "text",
    "text/plain"
  ],
  "defaultOutputModes": [
    "text",
    "json",
    "text/plain"
  ],
  "skills": [
    {
      "id": "read_github_repos",
      "name": "GitHub_Retriever",
      "description": "Fetches and analyzes recent content updates from GitHub repositories,
including open/closed issues, pull request statuses, and detailed commit logs.",
      "tags": [
        "Find the newest pull requests from",
        "Check recent issues in",
        "Summarize commits for",
        "GitHub repository status"
      ],
      "examples": [
        "Find the 10 most recent pull requests from the langchain-ai/langgraph repository
along with their commits.",
        "List all open issues tagged with 'bug' in the current repository."
      ]
    }
  ]
}
```

將下列內容複製到 `app/cards/content_evaluator_agent_card.json` 檔案中。這會向主機代理程式說明這個代理程式的功能、可提供的資料類型，以及代理程式會傳回的內容。

與先前相同，使用這個指令編輯評估人員代理程式卡片：

```
cloudshell edit $HOME/app/cards/content_evaluator_agent_card.json
```

```
{
  "name": "Content_Evaluation_Agent",
  "description": "Distills and summarizes complex GitHub data, breaking down pull requests,
code changes, and issue discussions into easily understandable insights.",
  "url": "http://localhost:8002",
  "capabilities": {
    "streaming": true,
    "pushNotifications": true,
    "stateTransitionHistory": false
  },
  "defaultInputModes": [
    "text",
    "json",
    "text/plain"
  ],
  "defaultOutputModes": [
    "text",
    "text/plain"
  ],
  "skills": [
    {
      "id": "evaluate_github_content",
      "name": "Evaluate GitHub Content",
      "description": "Analyzes and synthesizes provided GitHub data—including code diffs,
commit histories, issue threads, and PR comments—into clear, structured summaries.",
      "tags": [
        "summarize pull request",
        "evaluate GitHub changes",
        "break down commits",
        "distill issue discussion",
        "code review summary"
      ],
      "examples": [
        "Make a thorough summary of the code changes and commit history from this pull
request data.",
        "Break down the main arguments and proposed solutions from this provided GitHub issue
discussion."
      ]
    }
  ]
}
```

以下是電子郵件代理程式的代理資訊卡。使用相同的 Cloud Shell 指令，將其複製到 `app/cards/emailer_agent_card.json` 檔案：

```
cloudshell edit $HOME/app/cards/emailer_agent_card.json
```

```
{
  "name": "Email_Agent",
  "description": "Acts as the final delivery mechanism in the pipeline, dispatching generated
summaries and reports to designated email addresses.",
  "url": "http://localhost:8003",
  "capabilities": {
    "streaming": true,
    "pushNotifications": true,
    "stateTransitionHistory": false
  },
  "defaultInputModes": [
    "text",
    "text/plain"
  ],
  "defaultOutputModes": [
    "text",
    "text/plain"
  ],
  "skills": [
    {
      "id": "dispatch_email_report",
      "name": "Dispatch_Report_via_Email",
      "description": "Takes synthesized text data and securely emails it to a specified
recipient or distribution list.",
      "tags": [
        "send email",
        "email summary",
        "dispatch report",
        "forward to inbox"
      ],
      "examples": [
        "Email me the summarized evaluations from the retrieved pull request and issue
data.",
        "Take this code review breakdown and send it in an email to the dev team."
      ]
    }
  ]
}
```

步驟 7 · 約 1 分鐘

7. 建立遠端代理程式執行器

如要讓主機代理程式「呼叫」或「對話」遠端代理程式，每個代理程式都需要 A2A 伺服器可見的代理程式執行器，並由代理程式資訊卡識別。執行器是具有 execute() 和 cancel() 方法的抽象類別，可叫用遠端代理程式並提供工作或訊息，或直接取消工作。

以下是抽象 ADK 代理執行器的實作方式，可協助將訊息和排序工作傳送至遠端代理，所有項目都會交換 TextPart 構件。因此，我們可以在代理程式之間共用常見的代理程式執行器。建立新的執行器檔案：

```
touch $HOME/app/src/agents/executor.py
```

將下列程式碼片段複製到新的 `$HOME/app/src/agents/executor.py` 檔案中：

```
# $HOME/app/src/agents/executor.py
from a2a.server.agent_execution import AgentExecutor, RequestContext
from a2a.server.agent_execution.context import RequestContext
from a2a.server.events.event_queue import EventQueue
from a2a.server.tasks import TaskUpdater
from a2a.types import TaskState, TextPart, Part
from a2a.utils import new_agent_text_message, new_task

from google.adk.artifacts import InMemoryArtifactService
from google.adk.memory.in_memory_memory_service import InMemoryMemoryService
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

from google.genai import types

class BaseAgentExecutor(AgentExecutor):
    """An AgentExecutor that runs a remote ADK Agent"""
    def __init__(
            self,
            agent,
            status_message='Processing request...',
            artifact_name='response',
        ):
        """Initialize a generic ADK agent executor.

        Args:
            agent: The ADK agent instance
            status_message: Message to display while processing
            artifact_name: Name for the response artifact
        """
        self.agent = agent
        self.status_message = status_message
        self.artifact_name = artifact_name
        self.runner = Runner(
            app_name=agent.name,
            agent=agent,
            artifact_service=InMemoryArtifactService(),
            session_service=InMemorySessionService(),
            memory_service=InMemoryMemoryService(),
        )

    async def execute(
        self,
        context: RequestContext,
        event_queue: EventQueue,
    ) -> None:
        query = context.get_user_input()
        task = context.current_task or new_task(context.message) # type: ignore
        await event_queue.enqueue_event(task)

        updater = TaskUpdater(event_queue, task.id, task.context_id)
        if context.call_context:
```

```
            user_id = context.call_context.user.user_name
        else:
            user_id = "a2a_user"

        try:
            # Update status with custom message
            await updater.update_status(
                TaskState.working,
                new_agent_text_message(
                    self.status_message,
                    task.context_id,
                    task.id
                ),
            )

            # Process with ADK agent
            session = await self.runner.session_service.create_session(
                app_name=self.agent.name,
                user_id=user_id,
                state={},
                session_id=task.context_id,
            )

            content = types.Content(
                role="user", parts=[types.Part.from_text(text=query)]
            )

            response_text = ""
            async for event in self.runner.run_async(
                user_id=user_id, session_id=session.id, new_message=content
            ):
                if event.is_final_response() and event.content and event.content.parts:
                    for part in event.content.parts:
                        if hasattr(part, "text") and part.text:
                            response_text += part.text + "\n"
                        elif hasattr(part, "function_call"):
                            # Log or handle function calls if needed
                            pass  # Function calls are handled internally by ADK

            # Add response as artifact with custom name
            await updater.add_artifact(
                [Part(root=TextPart(text=response_text))],
                name=self.artifact_name,
            )

            await updater.complete()

        except Exception as e:
            await updater.update_status(
                TaskState.failed,
                new_agent_text_message(f"Error: {e!s}", task.context_id, task.id),
                final=True,
```

```
            )

    async def cancel(
            self,
            context: RequestContext,
            event_queue: EventQueue
        ) -> None:
        """Cancel the execution of a specific task."""
        raise NotImplementedError("Cancel not implemented for RetrieverAgentExecutor")
```

現在我們有了抽象代理程式執行器，就能使用 Pythonic 繼承功能，根據每個代理程式的特定功能和需求微調執行器。在本例中，呼叫代理程式並傳送酬載、等待回應及擷取回應酬載的執行作業，對工作流程而言是通用的。因此不需要進行任何特定變更。不過，每個代理程式 A2A 伺服器都需要代理程式執行器。執行下列指令，為所有三個代理程式建立執行器檔案：

```
touch $HOME/app/src/agents/retriever/executor.py
touch $HOME/app/src/agents/evaluator/executor.py
touch $HOME/app/src/agents/emailer/executor.py
```

現在，請在每個檔案中加入對應內容。

```
# $HOME/app/src/agents/retriever/executor.py
from ..executor import BaseAgentExecutor

class RetrieverAgentExecutor(BaseAgentExecutor):
    """
    An AgentExecutor that runs the Retriever Agent

    All agent specific implementations for execute() and cancel() can be
    overloaded here, along with any other desired funcitonality.
    """
    pass
```

```
# $HOME/app/src/agents/evaluator/executor.py
from ..executor import BaseAgentExecutor

class EvaluatorAgentExecutor(BaseAgentExecutor):
    """
    An AgentExecutor that runs the Evaluator Agent

    All agent specific implementations for execute() and cancel() can be
    overloaded here, along with any other desired funcitonality.
    """
    pass
```

```
# $HOME/app/src/agents/emailer/executor.py
from ..executor import BaseAgentExecutor

class EmailerAgentExecutor(BaseAgentExecutor):
    """
    An AgentExecutor that runs the Emailer Agent

    All agent specific implementations for execute() and cancel() can be
    overloaded here, along with any other desired funcitonality.
    """
    pass
```

步驟 8 · 約 1 分鐘

8. 向 A2A 伺服器公開遠端代理程式

現在每個代理程式都有自己的商家名片，透過 A2A 伺服器向端點公開後，就能被發現。為了方便進行本程式碼研究室，我們將整個程序保留在本機，並為每個遠端代理程式端點使用本機主機。但實際上，這些項目可以公開給任何所需端點，只要在代理程式資訊卡的 `url` 欄位中參照，就能探索到這些項目。

每個遠端代理程式的下一步，就是將其公開給自己的 A2A 伺服器。執行這項指令，為所有三個遠端代理程式建立 `server.py` 檔案：

```
touch $HOME/app/src/agents/retriever/server.py
touch $HOME/app/src/agents/evaluator/server.py
touch $HOME/app/src/agents/emailer/server.py
```

現在，您要為每個代理程式新增伺服器程式碼。從檢索器代理程式開始：

```
# $HOME/app/src/agents/retriever/server.py
import logging
import os
import json
import uvicorn

from a2a.types import AgentCard
from a2a.server.apps import A2AStarletteApplication
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.tasks import InMemoryTaskStore

from .agent import root_agent as retriever_agent
from .executor import RetrieverAgentExecutor

logging.basicConfig(level=logging.INFO)

with open("cards/github_retrieval_agent_card.json", "r") as f:
    card_data = json.load(f)
github_retrieval_agent_card = AgentCard(**card_data)

request_handler = DefaultRequestHandler(
    agent_executor=RetrieverAgentExecutor(
        agent=retriever_agent
    ),
    task_store=InMemoryTaskStore(),
)

server = A2AStarletteApplication(
    http_handler=request_handler,
    agent_card=github_retrieval_agent_card,
)

if __name__ == "__main__":
    port = int(os.getenv("PORT", 8001))
    print(f"Starting Retriever Agent on Port {port}...")
    uvicorn.run(server.build(), host="0.0.0.0", port=port)
```

接著，為評估代理程式建立伺服器：

```
# $HOME/app/src/agents/evaluator/server.py
import logging
import os
import json
import uvicorn

from a2a.types import AgentCard
from a2a.server.apps import A2AStarletteApplication
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.tasks import InMemoryTaskStore

from .agent import root_agent as evaluator_agent
from .executor import EvaluatorAgentExecutor

logging.basicConfig(level=logging.INFO)

with open("cards/content_evaluator_agent_card.json", "r") as f:
    card_data = json.load(f)
content_evaluator_agent_card = AgentCard(**card_data)

request_handler = DefaultRequestHandler(
    agent_executor=EvaluatorAgentExecutor(agent=evaluator_agent),
    task_store=InMemoryTaskStore(),
)

server = A2AStarletteApplication(
    http_handler=request_handler,
    agent_card=content_evaluator_agent_card,
)

if __name__ == "__main__":
    port = int(os.getenv("PORT", 8002))
    print(f"Starting Evaluator Agent on Port {port}...")
    uvicorn.run(server.build(), host="0.0.0.0", port=port)
```

最後，針對電子郵件代理程式：

```
# $HOME/app/src/agents/emailer/server.py
import logging
import os
import json
import uvicorn

from a2a.types import AgentCard
from a2a.server.apps import A2AStarletteApplication
from a2a.server.request_handlers import DefaultRequestHandler
from a2a.server.tasks import InMemoryTaskStore

from .agent import root_agent as emailer_agent
from .executor import EmailerAgentExecutor

logging.basicConfig(level=logging.INFO)

with open("cards/emailer_agent_card.json", "r") as f:
    card_data = json.load(f)
emailer_agent_card = AgentCard(**card_data)

request_handler = DefaultRequestHandler(
    agent_executor=EmailerAgentExecutor(
        agent=emailer_agent
    ),
    task_store=InMemoryTaskStore(),
)

server = A2AStarletteApplication(
    http_handler=request_handler,
    agent_card=emailer_agent_card,
)

if __name__ == "__main__":
    port = int(os.getenv("PORT", 8003))
    print(f"Starting Emailer Agent on Port {port}...")
    uvicorn.run(server.build(), host="0.0.0.0", port=port)
```

步驟 9 · 約 2 分鐘

9. 建構主機代理程式

我們目前所做的，基本上是將每個代理程式都變成主機代理程式可 Ping 的 API。現在遠端代理程式已連上自己的 A2A 伺服器，我們需要建構主機代理程式，以便探索及協調這些代理程式。

為達成此目標，我們需要建構主機的幾個不同層面。首先，我們需要為每個遠端代理程式及其伺服器建立個別用戶端。方法是建立 A2A 用戶端 Factory，與伺服器代管的每個遠端代理程式端點建立連線，主機即可探索代理卡。主機與每個遠端代理程式之間的連線，都可以使用 RemoteAgentConnections 物件初始化。

```
touch $HOME/app/src/host/remote_agent_connection.py
touch $HOME/app/src/host/agent.py
touch $HOME/app/src/host/__init__.py
echo "from . import agent" >> $HOME/app/src/host/__init__.py
```

將下列遠端代理程式連線實作複製到 `src/host/remote_agent_connection.py` 檔案中：

```python
# src/host/remote_agent_connection.py
import traceback

from a2a.client import (
    Client,
    ClientFactory,
)
from a2a.types import (
    AgentCard,
    Message,
    Task,
    TaskState,
)


class RemoteAgentConnection:
    """A class to hold the connections to the remote agents."""

    def __init__(self, client_factory: ClientFactory, agent_card: AgentCard):
        self.agent_client: Client = client_factory.create(agent_card)
        self.card: AgentCard = agent_card

    def get_agent(self) -> AgentCard:
        return self.card

    async def send_message(self, message: Message) -> Task | Message | None:
        lastTask: Task | None = None
        try:
            async for event in self.agent_client.send_message(message):
                if isinstance(event, Message):
                    return event
                if self.is_terminal_or_interrupted(event[0]):
                    return event[0]
                lastTask = event[0]
        except Exception as e:
            print('Exception found in send_message')
            traceback.print_exc()
            raise e
        return lastTask

    def is_terminal_or_interrupted(self, task: Task) -> bool:
        return task.status.state in [
            TaskState.completed,
            TaskState.canceled,
            TaskState.failed,
            TaskState.input_required,
            TaskState.unknown,
        ]
```

我們現在可以透過指定的用戶端和代理商卡片，在用戶端和伺服器之間建立連線。主機代理程式會使用這項工具，與遠端代理程式伺服器建立連線。

接著來建立主機代理程式。首先，請將下列程式碼片段複製到 `app/src/host/agent.py` 檔案中。這是 Pythonic 類別的初始化作業，需要遠端代理程式端點和標準 httpx 用戶端。初始化這個代理程式類別時，系統會透過提供給代理程式的遠端位址建立連線。

```
import asyncio
import json
import os
import uuid
import httpx
from typing import Any

from a2a.client import A2ACardResolver, ClientConfig, ClientFactory
from a2a.types import (
    AgentCard,
    Message,
    Part,
    Role,
    Task,
    TaskState,
    TextPart,
    TransportProtocol,
)

from google.adk import Agent
from google.adk.agents.callback_context import CallbackContext
from google.adk.agents.readonly_context import ReadonlyContext
from google.adk.tools.tool_context import ToolContext
from dotenv import load_dotenv

from .remote_agent_connection import RemoteAgentConnection

load_dotenv()


####################################
# --- Coordinator Agent Definition ---
####################################
class CoordinatorAgent:
    """
    The Coordinator agent.
    This is the agent responsible for sending tasks to agents.
    """

    def __init__(
        self,
        remote_agent_addresses: list[str],
        http_client: httpx.AsyncClient,
    ):
        self.http_client = http_client
        self.remote_agent_addresses = remote_agent_addresses
        self.client_factory = None
        self.remote_agent_connections: dict[str, RemoteAgentConnection] = {}
        self.cards: dict[str, AgentCard] = {}
        self.agents: str = ''
```

主機代理程式的下一個層面是建立與遠端代理程式的連線。`ensure_initialized` 方法不會在 `__init__` 中連線 (這會在模組匯入時執行，事件迴圈存在之前)，而是會將這項工作延後到實際使用代理程式時。這個函式會檢查是否已建立連線，如果沒有，則會建立 A2A 用戶端，並與每個遠端代理程式平行連線。在 `app/src/host/agent.py` 中，將這個區隔複製到上一個區隔的正下方。

```
    ####################################
    # --- Remote Agent Initialization ---
    ####################################
    async def ensure_initialized(self):
        if not self.remote_agent_connections:
            config = ClientConfig(
                httpx_client=self.http_client,
                supported_transports=[
                    TransportProtocol.jsonrpc,
                    TransportProtocol.http_json,
                ],
            )
            self.client_factory = ClientFactory(config=config)
            async with asyncio.TaskGroup() as task_group:
                for address in self.remote_agent_addresses:
                    task_group.create_task(self.retrieve_card(address))

    async def retrieve_card(self, address: str):
        card_resolver = A2ACardResolver(self.http_client, address)
        card = await card_resolver.get_agent_card()
        card.url = address  # Use the actual address, not the hardcoded URL in the card JSON

        remote_connection = RemoteAgentConnection(self.client_factory, card)
        self.remote_agent_connections[card.name] = remote_connection
        self.cards[card.name] = card

        agent_info = []
        for card in self.cards.values():
            agent_info.append(
                json.dumps({"name": card.name, "description": card.description})
            )
        self.agents = "\n".join(agent_info)
```

接著是 LLM 代理程式實作。在本例中，我們使用 ADK 代理做為協調器，因此需要提示指令、回呼和工具等一般外掛程式。將所有這些元件複製到 `app/src/host/agent.py` 檔案中的主機代理程式類別內。將下列回呼放入主機代理程式類別。在代理程式正確觸發之前，這個回呼會初始化代理程式 (如果代理程式狀態尚未初始化)。

```
    ############################
    # -- Implement the ADK Agent
    ############################

    # --- Before Model Callback ---
    def before_model_callback(
        self, callback_context: CallbackContext, llm_request
    ):
        """
        A callback to set up the session state before the model processes the
        request
        """
        state = callback_context.state
        if 'session_active' not in state or not state['session_active']:
            if 'session_id' not in state:
                state['session_id'] = str(uuid.uuid4())
            state['session_active'] = True
```

主機代理的下一個元件是提示指令。由於這是我們的自動調度管理工具代理程式，因此需要知道可供使用的遠端代理程式，以及目前處於活動狀態的代理程式，讓協調器瞭解工作階段的現況。在回呼下方新增下列提示和輔助函式。

```
    # --- Prompt ---
    def root_instruction(self, context: ReadonlyContext) -> str:
        current_agent = self.check_state(context)
        return f"""
            You are an expert orchestrator that can delegate user requests to the
            appropriate remote agents to generate a GitHub research report.

            **Your Goal:** To fulfill user requests for GitHub repository data, evaluate it,
and optionally email a report.

            **Workflow Steps:**

            1.  **Understand the User Request**:
                -   Identify repository names (e.g., "google/adk-python").
                -   Determine if the user wants Pull Requests, Issues, or both.
                -   Extract any specified email address for sending the report (e.g.,
"user@example.com").
                -   Note the number of PRs/issues requested per repository. If not specified,
the Retrieval Agent has defaults.

            2.  **Retrieve Data (GitHub_Retrieval_Agent)**:
                -   Use the `send_message` tool to send a message to the
"GitHub_Retrieval_Agent".
                -   Your message to the Retrieval Agent should clearly state which
repositories to fetch data for, and specify if you need issues, pull requests, and the
respective limits.
                -   Example message to Retrieval Agent: "Fetch 5 issues and 3 pull requests
for google/adk-python."

            3.  **Evaluate and Summarize Data (Content_Evaluation_Agent)**:
                -   Once you receive the raw JSON data from the "GitHub_Retrieval_Agent", use
the `send_message` tool to send this data to the "Content_Evaluation_Agent".
                -   Your message to the Evaluation Agent should include the raw JSON data you
received.
                -   The Evaluation Agent will return a Markdown-formatted report.

            4.  **Email Report (Email_Agent - if email provided)**:
                -   If the user's initial request included an email address, use the
`send_message` tool to send the Markdown report from the "Content_Evaluation_Agent" to the
"Email_Agent".
                -   Your message to the Email Agent should include the report and the
recipient's email address.
                -   Example message to Email Agent: "Send this report to user@example.com:
[Markdown Report Content]".

            5.  **Respond to User**:
                -   Based on the outcome of the steps above, formulate a concise and
informative response to the user.
                -   If a report was generated and emailed, confirm that. If only a report was
generated, provide it directly to the user.
                -   If an email address was requested but the email failed to send, inform
the user.
```

```
                -   If any step failed, inform the user about the failure.

            **Available Tools:**

            -   `list_remote_agents()`: Use this to see what agents are available (though you
already know their names for this workflow).
            -   `send_message(agent_name: str, message: str)`: Use this to interact with
remote agents.

            **Crucial Guidelines:**

            -   **Rely on Tools**: ALWAYS use `send_message` to interact with the remote
agents. Do NOT attempt to perform the tasks yourself.
            -   **No Conversational Filler**: Only communicate the essential information to
the remote agents or back to the user.
            -   **Error Handling**: If a remote agent returns an error, acknowledge it and
try to provide a helpful message to the user.

            Agents:
            {self.agents}

            Current agent: {current_agent['active_agent']}
        """

    def check_state(self, context: ReadonlyContext):
        state = context.state
        if (
            'context_id' in state
            and 'session_active' in state
            and state['session_active']
            and 'agent' in state
        ):
            return {'active_agent': f'{state["agent"]}'}
        return {'active_agent': 'None'}
```

現在要介紹主機代理程式最重要的部分：工具。主機將可存取 send_message() 工具和 list_remote_agents() 工具。list_remote_agent() 工具較為簡單，只會讀取類別的代理程式資訊卡，並傳回包含每個代理程式名稱和說明的字典清單。send_message() 稍微複雜一點。只要提供代理程式名稱和訊息字串，即可將訊息或工作 (串流或非串流) 傳送給遠端代理程式。將這段程式碼放在同一個 Host Agent Python 類別中，位於 `app/src/host/agent.py` 的提示詞部分下方。

```
    # --- Agent Tools ---
    def list_remote_agents(self):
        """List the available remote agents you can use to delegate the task."""
        if not self.remote_agent_connections:
            return []

        remote_agent_info = []
        for card in self.cards.values():
            remote_agent_info.append(
                {'name': card.name, 'description': card.description}
            )
        return remote_agent_info

    async def send_message(
        self, agent_name: str, message: str, tool_context: ToolContext
    ):
        """Sends a task either streaming (if supported) or non-streaming.

        This will send a message to the remote agent named agent_name.

        Args:
          agent_name: The name of the agent to send the task to.
          message: The message to send to the agent for the task.
          tool_context: The tool context this method runs in.

        Yields:
          A dictionary of JSON data.
        """
        await self.ensure_initialized()
        if agent_name not in self.remote_agent_connections:
            raise ValueError(f'Agent {agent_name} not found')
        state = tool_context.state
        state['agent'] = agent_name

        client = self.remote_agent_connections[agent_name]
        if not client:
            raise ValueError(f'Client not available for {agent_name}')

        task_id = state.get('task_id', None)
        context_id = state.get('context_id', None)
        message_id = state.get('message_id', None)
        task: Task
        if not message_id:
            message_id = str(uuid.uuid4())

        request_message = Message(
            role=Role.user,
            parts=[Part(root=TextPart(text=message))],
            message_id=message_id,
            context_id=context_id,
            task_id=task_id,
        )
```

```
        response = await client.send_message(request_message)

        if isinstance(response, Message):
            return await convert_parts(response.parts, tool_context)

        task: Task = response # type: ignore

        # Assume completion unless a state returns that isn't complete
        state['session_active'] = not client.is_terminal_or_interrupted(task)
        if task.context_id:
            state['context_id'] = task.context_id
        state['task_id'] = task.id

        if task.status.state == TaskState.input_required:
            # Force user input back
            tool_context.actions.skip_summarization = True
            tool_context.actions.escalate = True
        elif task.status.state == TaskState.canceled:
            # Open question, should we return some info for cancellation instead
            raise ValueError(f'Agent {agent_name} task {task.id} is cancelled')
        elif task.status.state == TaskState.failed:
            # Raise error for failure
            raise ValueError(f'Agent {agent_name} task {task.id} failed')

        response = []
        if task.status.message:
            response.extend(
                await convert_parts(task.status.message.parts, tool_context)
            )

        if task.artifacts:
            for artifact in task.artifacts:
                response.extend(
                    await convert_parts(artifact.parts, tool_context)
                )
        return response
```

最後，我們將實作 ADK 代理程式，整合所有這些元件。這是 Orchestrator 類別代理程式的大腦，會在 BeforeModelCallback 執行後，使用可用的工具執行收到的提示。

將下列程式碼片段放在 `app/src/host/agent.py` 檔案的工具區段下方：

```
    def create_agent(self) -> Agent:
        """Create an instance of the CoordinatorAgent."""
        return Agent(
            model='gemini-2.5-flash',
            name='host',
            instruction=self.root_instruction,
            before_model_callback=self.before_model_callback,
            description=(
                'This coordinator agent orchestrates the retriever, evaluator, and emailer
agents'
            ),
            tools=[
                self.send_message,
                self.list_remote_agents,
            ],
        )
```

代理程式需要幾個輔助函式，才能使用 send_message() 工具將 A2A 部分轉換為原始文字和資料。在 CoordinatorAgent 類別「外部」複製這個區隔：

```
##########################
# --- Helper Functions ---
##########################
async def convert_part(part: Part, tool_context: ToolContext):
    """Convert a part to text. Only text parts are supported."""
    if isinstance(part.root, TextPart):
        return part.root.text
    if part.root.kind == "data":
        return part.root.data
    return f"Unknown type: {part}"

async def convert_parts(parts: list[Part], tool_context: ToolContext):
    """Convert parts to text."""
    rval = []
    for p in parts:
        rval.append(await convert_part(p, tool_context))
    return rval
```

為方便透過 `uv run adk web` 或 `adk run` 進行本機測試和互動，請在 `app/src/host/agent.py` 檔案底部新增 `root_agent` 初始化作業：

```
root_agent = CoordinatorAgent(
    remote_agent_addresses=[
        os.getenv('RETRIEVAL_AGENT_URL', 'http://localhost:8001'),
        os.getenv('EVALUATOR_AGENT_URL', 'http://localhost:8002'),
        os.getenv('EMAILER_AGENT_URL', 'http://localhost:8003'),
    ],
    http_client=httpx.AsyncClient(timeout=30),
).create_agent()
```

恭喜！您已建立第一個 A2A 主機代理程式。由於我們已使用 root_agent 變數初始化，因此可以透過本機 uv 執行 adk 網頁部署作業與其互動，就像其他遠端代理程式一樣。請使用下列指令與主機互動：

```
cd $HOME/app/src/

uv run adk run host
```

步驟 10 · 約 1 分鐘

10. 在本機測試

如要測試完整的多代理系統，您必須啟動要讓主機代理程式使用的遠端代理伺服器。這會一次啟動所有代理程式伺服器。

```
# Make sure you have activated your virtual environment first!
# source .venv/bin/activate

#!/bin/bash

# Exit immediately if a command exits with a non-zero status.
set -e

# Function to kill all background processes
cleanup() {
    echo "Caught signal, terminating background processes..."
    # The negative PID kills the entire process group
    kill -TERM -$$
    wait
    echo "All processes terminated."
    exit
}

# Trap TERM, INT, and EXIT signals and call the cleanup function
trap cleanup TERM INT EXIT

# Activate virtual environment
uv sync

# Start the agent servers in the background
echo "Starting agent servers..."
uv run python3 -m src.agents.retriever.server --port 8001 &
uv run python3 -m src.agents.evaluator.server --port 8002 &
uv run python3 -m src.agents.emailer.server --port 8003 &

cd $HOME/app/src/
RETRIEVER_AGENT_URL=http://localhost:8001
EVALUATOR_AGENT_URL=http://localhost:8002
EMAILER_AGENT_URL=http://localhost:8003
uv run adk run host

# Wait for all background processes to finish
wait
```

這個 Bash 指令會啟動所有四個代理程式伺服器 (擷取器、評估器、電子郵件傳送器和主機)。終端機中會顯示每個伺服器的記錄輸出內容。伺服器執行完畢後，請開啟**新的終端機視窗** (請確認您位於相同的 `app` 目錄，且虛擬環境已啟用)，然後啟動 uv 執行 ADK 網頁介面：

```
# In a new terminal, from the 'app' directory
source .venv/bin/activate
cd $HOME/app/src/
uv run adk web
```

開啟終端機中顯示的 localhost 連結。選取左上方的下拉式選單，即可直接與主機代理程式互動。

步驟 11 · 約 1 分鐘

11. 清除

如要避免持續產生費用，請刪除在本程式碼研究室中建立的資源。

停止所有正在執行的代理程式伺服器，然後刪除專案 (如果您是特地為這個程式碼研究室建立專案)。前往啟動遠端代理程式的終端機，執行 `Ctrl+C`，然後移除這個 Google Cloud 專案：

```
gcloud projects delete ${GOOGLE_CLOUD_PROJECT}
```

步驟 12 · 約 1 分鐘

12. 恭喜

您已使用 Agent2Agent 成功建構多代理系統！

目前所學內容

- 如何建立具有專屬工具的獨立 ADK 代理
- 如何使用代理資訊卡，讓代理擁有可探索的身分
- 如何透過 A2A 伺服器公開代理程式
- 如何建構可自動調度遠端代理程式的主機代理程式
- A2A 通訊協定如何讓獨立開發的代理進行通訊

後續步驟

- 將系統部署至 Cloud Run 以用於實際工作環境
- 新增更多遠端代理程式，擴充系統功能
- 瞭解 A2A 的串流和推播通知功能

參考文件

- ADK 說明文件
- Agent2Agent 通訊協定